

แบบฝึกหัดที่ 2

1. วิเคราะห์เวลาการทำงานของส่วนของอัลกอริทึมต่อไปนี้ และสรุปให้อยู่ในรูปของสัญกรณ์เชิงเส้นกำกับ (แสดงขั้นตอนการวิเคราะห์ด้วย)

ข้อ	โจทย์	คำตอบ
1	$a = 0$ $a = a + 1$	ทำครั้งเดียว <u>ตอบ</u> $O(1)$
2	$s = 0$ for($i=1$ to n) $s = s + i$	ทำ n รอบ <u>ตอบ</u> $O(n)$
3	for($i=1$ to n) for($j=1$ to n) $data[i][j] = 0$	loop นอก n รอบ loop ใน n รอบ <u>ตอบ</u> $O(n^2)$
4	$s = 0$ for($i=1$ to n) for($j=i+1$ to n) for($k=1$ to j) $s = s + 1$	loop นอกถึง n รอบ loop กลางถึง n loop ในถึง $j \rightarrow j = i+1$ ถึง n <u>ตอบ</u> $O(n^3)$
5	$s = 0$ while($n > 0$) $n = \lfloor n/3 \rfloor$ $s = s + 1$	n ลดลงด้วยหาร 3 $\rightarrow \log \rightarrow$ ทำ $\log_3 n$ รอบ <u>ตอบ</u> $O(\log n)$
6	Dummy(n) { if($n \leq 0$) return -1 ① for($i=1$ to n) $s = s + 1$ ② for($i=1$ to 2) Dummy($n/2$) }	① ทำงาน n รอบ ② call Dummy($n/2$) ทำ 2 รอบ $T(n) = 2T(n/2) + n$ จะได้ $2T(n/2)$ Master Method: $a=2, b=2, c=\log_2 a = \log_2 2 = 1$ จะได้ $k=c \rightarrow \Theta(n^k \log n)$ <u>ตอบ</u> $\Theta(n \log n)$

2. แสดงวิธีการหาผลเฉลี่ยของข้อย่อยต่อไปนี้ และตอบให้อยู่ในรูป $\Theta(g(n))$ โดยฟังก์ชัน $g(n)$ เป็นฟังก์ชันที่มีความซับซ้อนน้อยที่สุด

```

1: Algorithm1(a[1..n], n){
2:   for( j=1; j<n ;j++){
3:     for(i=1; i<n ;i++ ){
4:       if(a[i] > a[i+1]){
5:         tmp = a[i];
6:         a[i] = a[i+1];
7:         a[i+1]=tmp;
8:       }
9:     }
10:  }
11: }
    
```

Outer loop : for(j=1; j<n; j++) ทำงาน $n-1$ รอบ

Inner loop : for(i=1; i<n; i++) ทำงาน $n-1$ รอบ

if กับ swap ใช้เวลา $O(1)$ รอบ

$$\text{Sum} = (n-1)(n-1) \approx n^2$$

ดังนั้น $g(n) = n^2$ และ Time complexity เท่ากับ $\Theta(n^2)$ ~~✗~~

3. พิจารณาอัลกอริทึมข้อ 2.1, 2.2 และ 2.3 ให้วิเคราะห์เวลาการทำงานของอัลกอริทึมทั้ง 3 ซ้อย่อย สรุปให้อยู่ในรูปสัญกรณ์เชิงเส้นกำกับ แล้วสรุปว่าอัลกอริทึมใดมีประสิทธิภาพในเชิงเวลามากกันพร้อมเหตุผล

3.1 อัลกอริทึมด้านล่างใช้เวลาเป็น $\Theta(n^2)$

อัลกอริทึม	แสดงวิธีการวิเคราะห์เวลา
<pre> ALGO1(a[1..n]){ for(i=1 to n){ c = 0 for(j=1 to n) if(a[i]==a[j]) c = c+1 if(c>n/2) return true } return false } </pre>	loop นอก = n รอบ loop ใน = n รอบ ค่าที่ซ้ำใน = $O(n)$ Sum = $n \times n = n^2$ $\therefore$ Time complexity is $\Theta(n^2)$

3.2 อัลกอริทึมด้านล่างใช้เวลาเป็น $\Theta(n \log n)$

อัลกอริทึม	แสดงวิธีการวิเคราะห์เวลา
<pre> ALGO2(a[1..n]){ If (n==1) return true mergeSort(a) d = a[1], c=1; for(i=2 to n){ if(d==a[i]) c = c + 1 else d = a[i], c=1 if(c>n/2) return true } return false } </pre>	mergeSort(a) = $\Theta(n \log n)$ Loop = $\Theta(n)$ จะได้ $T(n) = n \log n + n$ $\therefore$ time complexity is $\Theta(n \log n)$

หมายเหตุ ให้เมธอด mergeSort(a) ใช้เวลา $\Theta(n \log n)$

3.3 อัลกอริทึมด้านล่างใช้เวลาเป็น $\Theta(n)$

อัลกอริทึม	แสดงวิธีการวิเคราะห์เวลา
<pre> ALGO3(a[1..n]){ c=0 m=findM(a) for(i=1 to n) if(m == a[i]) c = c + 1 if(c>n/2) return true }</pre>	$m = \text{findM}(n) = O(n)$ $\text{loop} + \text{condition} = O(n)$ จะได้ $T(n) = n + n = 2n$ $\therefore$ Time complexity is $\Theta(n)$

หมายเหตุ ให้เมธอด findM(a) ใช้เวลา $O(n)$

3.4 เปรียบเทียบประสิทธิภาพเชิงเวลาของอัลกอริทึมทั้ง 3 พร้อมให้เหตุผล

สามารถเรียงลำดับตามประสิทธิภาพได้อดังนี้ $ALGO3 > ALGO2 > ALGO1$ เนื่องจาก

$ALGO3$ ใช้เวลา $\Theta(n)$ ซึ่งเติบโตช้าที่สุด ซึ่งสามารถเรียงลำดับการเติบโตได้จากน้อยสุดไปมากที่สุดได้อดังนี้

$\Theta(n) < \Theta(n \log n) < \Theta(n^2)$

$ALGO3 \quad ALGO2 \quad ALGO1$